uniRAG: An Offline Retrieval-Augmented Study Assistant with Deterministic Exam Analytics for University Students

Raghavendra Chandrakantha Bhat
Department of Computer Science and Applications, Bangalore University, Bangalore, India
raghavendra.c.bhat@gmail.com

Abstract—Examination preparation at universities depends on scattered notes and scanned question papers, and students cannot easily learn how often a topic was asked, in which years or for how many marks. Cloud chatbots do not solve this, because they need the internet, send questions to external servers, ignore the university syllabus and can invent facts such as frequency counts. This paper presents uniRAG, an offline study assistant for the Master of Computer Applications programme of Bangalore University that runs on a single CPU-only laptop. It combines retrieval-augmented generation, a locally hosted Mistral 7B model, per-subject FAISS indexes and Tesseract OCR with a question bank of 1,426 questions extracted from papers of 18 subjects across seven exam years (2020–2026). The central design principle is that the language model writes, while deterministic Python code counts and checks: all frequencies, trends, repeated questions and high-weightage topics are computed from the question bank, and a calibrated relevance cut-off and syllabus gate stop the model from answering out-of-syllabus questions. On a labelled set of 61 queries the question matcher reached an F1 score of 0.757 at an empirically calibrated cut-off of 0.75, compared with 0.435 for standard retrieval. Across 125 repeated questions, a model answer bundle combines 67 curated answers with 58 offline model-generated answers, giving students an accurate study companion with zero running cost.

Index Terms—*retrieval-augmented generation, offline study assistant, exam intelligence, question extraction, language model hallucination, university education.*

I. INTRODUCTION

India has one of the largest higher-education systems in the world, with more than 43 million students enrolled across more than 1,100 universities and 45,000 colleges [1]. In professional programmes such as the Master of Computer Applications (MCA), preparing for semester examinations requires students to master both theoretical foundations and practical laboratory programming across five to six subjects per semester. The primary study materials are university-prescribed textbooks, teacher-provided lecture notes and past university question papers.

In practice, however, these materials are fragmented. Lecture notes are distributed as multi-megabyte PDF files, while question papers are circulated as scanned documents of

varying visual quality. When preparing for an examination, a student needs answers to three distinct questions: (1) what does this concept mean, (2) how has it been examined in previous years (its frequency, marks allotment and trend) and (3) what does an examination-standard answer look like? Answering the second and third questions today requires hours of manual searching through paper archives.

Large language models (LLMs) can explain almost any computer-science topic, but general-purpose chatbots such as ChatGPT, Claude or Gemini have four serious drawbacks for university exam preparation. First, they require an active internet connection, which is not always available or reliable in university hostels and rural areas. Second, they send every question and document to external cloud servers, raising privacy and data-ownership concerns. Third, because they are trained on web-scale text, they often answer outside the university's prescribed syllabus, introducing terminology or methods that the student's evaluators do not recognise. Fourth, and most crucially, LLMs hallucinate numbers and counts: when asked “how many times has Quick Sort appeared in Bangalore University MCA examinations?”, a cloud chatbot produces a plausible-sounding number that is almost invariably invented.

Retrieval-augmented generation (RAG) grounds an LLM's answers in a specific document store [2], [3], but a RAG system that lets the language model count occurrences from retrieved text still undercounts or invents frequencies, because retrieval returns only top-k chunks and language models are poor accountants [4], [5].

This paper describes uniRAG, built for the first three semesters of the MCA programme of Bangalore University. The system runs entirely offline on a single consumer laptop without a dedicated GPU. Its primary contributions are:

- **A fully offline RAG architecture** (Python, Streamlit, Ollama with Mistral 7B, nomic-embed-text, FAISS and Tesseract OCR) that operates with zero network access and zero per-query cost.
- **A model writes, code counts design:** all frequencies, trends, repeated questions and high-weightage topics are computed from a structured question bank by deterministic code; the language model is used only for conceptual explanation and analogy.
- **A four-level hybrid question matcher** and a set of negative guards (including an out-of-syllabus gate) that reject out-of-syllabus queries and maintain grounded answers.
- **An evaluation on the real university material:** question-matching cut-off calibration over 61 queries, a 29-case regression test harness, response-time profiling and a repeated-question solutions bundle covering 125 repeated questions across 16 subjects.

The rest of the paper is organised as follows. Section II reviews recent related literature. Section III describes the system architecture. Section IV details the implementation. Section V presents the methodology and guards. Section VI describes the experimental setup. Section VII reports results and discussion. Section VIII discusses limitations and threats to validity, and

Section IX concludes the paper with directions for future work.

II. LITERATURE REVIEW OF RECENT RESEARCH

This section reviews fifteen recent research works, published between 2023 and 2026, on retrieval-augmented generation, hallucination mitigation, offline LLM deployments, exam and educational chatbots, OCR quality in RAG pipelines, corpus-level reasoning and Indian-language retrieval.

A. Foundations: RAG and hallucination

RAG combines a retriever with a generator so that the model conditions its answer on retrieved text rather than internal parameters alone [2]. Gao et al.'s comprehensive survey divides RAG systems into naive, advanced and modular forms and describes retrieval, generation and augmentation techniques together with evaluation methods [3]. Another survey classifies LLM hallucinations into factuality and faithfulness errors, traces their causes to data, training and inference, and notes that even retrieval-augmented models still hallucinate and that the limits of retrieval need to be guarded against [4]. Ji et al. reached similar conclusions for text generation [5]. These works motivate uniRAG's choice of RAG over fine-tuning and show why code-level guards are necessary.

B. Improving and evaluating RAG

Self-RAG trains a language model to decide when to retrieve and critique its own output using special reflection tokens, improving factuality and citation accuracy over standard RAG [6]. Es et al. introduced RAGAS, an automated evaluation framework measuring faithfulness, answer relevance and context precision without human annotations [7]. Li et al. compared RAG with long-context LLMs across four tasks and about 19,000 questions: long-context models generally outperform RAG on synthesis, but RAG is far cheaper and faster for retrieval from large corpora, especially for exact-fact questions, so the two approaches are complementary [8].

C. RAG for education

Lang and Gurpinar built a chatbot that answered student questions in a self-paced online R-programming course and reported high student satisfaction and reduced teacher workload [9]. Kahl et al. compared retrieval-augmented with fine-tuned open-source models for a university robotics lecture and found that RAG improved ROUGE-1 scores from 0.22 to 0.43, outperforming fine-tuning on domain accuracy [10]. Ostrowska et al. integrated a RAG tutoring system into Moodle; it reached a faithfulness of 0.97 for STEM subjects, and students who used it achieved higher exam scores [11]. All three educational systems answer questions about course content. None of them, however, extract past exam papers or answer questions about exam history or frequency.

D. Offline and local RAG

Takamura and Umezawa built PRISM, an offline RAG system for regenerative-medicine questions that runs a 12B-parameter Gemma 3 model and a FAISS index on a laptop with Apple silicon; in thirty human-evaluated answer trials, binary accuracy rose from 60 % without RAG to 86.7 % with RAG, and latency was 3 to 6 s [12]. Lee deployed an offline RAG system on CPU-only laptops for natural-disaster advice using a 1B-parameter model; prompt caching and an on-disk index reduced generation latency from 60 s to 5–9 s [13]. Devine's ALoFTRAG showed that locally fine-tuning an embedder on the target corpus improved retrieval citation accuracy by 8.3 % and answer accuracy by 3.0 % on a consumer laptop [14]. Dayarathne et al. evaluated four open models, including Mistral-7B-Instruct, in a RAG pipeline over 4,929 computer-science abstracts: GPT-3.5 with RAG was best (0.905 accuracy on binary questions), Mistral-7B-Instruct was the best open model (0.857 accuracy), and Llama-2-7B was less reliable [15]. These works establish that an open 7B model can deliver useful answers on consumer hardware.

E. Counting and corpus-level questions

A question such as “how many times has this topic appeared in past exams?” requires aggregating over the entire collection of papers. Standard RAG cannot answer this: the retriever returns only top-k chunks, so the language model never sees the whole corpus. Luo et al. showed on their Global-RAG benchmark that standard RAG achieves only 1.51 F1 on corpus-level questions and that community-summary approaches such as GraphRAG are needed [16]. Zhu et al. demonstrated on Agri-Bench that top-k RAG undercounts on aggregation queries and proposed an agentic framework that decomposes the query and scans all chunks [17]. In uniRAG, we solve this without an agent loop: counts are not computed by reading retrieved chunks, but computed by code over a structured question bank.

F. OCR quality and RAG

Scanned papers are the main source of uniRAG's question bank, so OCR errors matter. Zhang et al. introduced OHRBench, with 8,561 document images and 8,498 question-answer pairs across seven RAG domains, and showed that the best OCR solutions still trail ground-truth text by at least 14 %, that OCR errors degrade both dense retrieval and answer generation, and that higher embedding dimensionality provides some robustness [18]. Sun et al. confirmed with InduOCRBench that high OCR accuracy does not guarantee high RAG accuracy: one model reached 82.9 % OCR accuracy but only about 53 % RAG accuracy, and domain-specific OCR fine-tuning helped [19]. In uniRAG, question-paper PDFs are scanned; OCR noise is mitigated by regular-expression pre-processing, character-level fuzzy matching and manual verification of high-frequency subjects.

G. Indian-language retrieval

Students in India often mix English with Hindi or Kannada words in their queries (e.g. “stack implementation ka program” or “queue kitni baar aaya”). Prasanjith et al. created IndicRAGSuite, evaluating RAG across eleven Indian languages, and found that multilingual embedding models such as BGE-M3 and Sarvam outperformed monolingual ones in eight out of thirteen language pairs [20]. In uniRAG, the query-understanding module handles Hinglish and Kannada transliterations by translating or mapping them to syllabus topics before retrieval.

H. Research gap

Table I summarises the reviewed works. Education-oriented systems focus on answering content questions, offline systems focus on general or clinical tasks, and corpus-level reasoning systems use heavy graph or agent frameworks. None combines offline RAG on consumer hardware with deterministic exam intelligence (counts, trends and priority) extracted from a structured question bank of real university papers. uniRAG addresses this gap.

TABLE I. Summary of reviewed work and uniRAG.

Work	Focus	Local / offline	Key result
[6] Asai, 2024	Self-reflective RAG	No	Better factuality, citations
[8] Li, 2024	Long context vs RAG	No	Complementary trade-offs
[9] Lang, 2025	RAG chatbot, R course	Not stated	High satisfaction
[10] Kahl, 2024	Tutor: RAG, fine-tuning	Partly	ROUGE-1 0.22 → 0.43
[11] Ostrowska, 2026	Moodle tutor	Not stated	Faithfulness 0.97
[12] Takamura, 2025	Medical RAG (PRISM)	Yes, laptop	Accuracy 60 → 86.7 %
[13] Lee, 2026	Disaster advice	Yes, CPU	Latency 60 s → 5–9 s
[14] Devine, 2025	Local fine-tuning	Yes	Answer acc. +3.0 %
[15] Dayarathne, 2025	Open 7B vs GPT-3.5	Open models	0.857 vs 0.905 acc.
[16] Luo, 2025	Counting in RAG	No	Standard RAG 1.51 F1
[17] Zhu, 2026	Aggregation queries	No	Top-k RAG undercounts
[18] Zhang, 2025	OCR impact on RAG	–	Up to ~50 % drop
[19] Sun, 2026	OCR robustness	–	OCR acc. ≠ RAG acc.
[20] Prasanjith, 2025	Indian-language RAG	–	BGE-M3 best in 8/13
uniRAG (this work)	**Study help + exam statistics**	**Yes, CPU laptop**	**F1 0.435 → 0.757**

III. SYSTEM ARCHITECTURE

uniRAG has four layers (Fig. 1). The presentation layer is a Streamlit application with role-based login (student and administrator). The application layer contains the query parser, question matcher, RAG pipeline, exam analytics, question paper solver and repeated-questions bundle generator. The local AI services are two Ollama models, Mistral 7B [23] for generation and parsing and nomic-embed-text [22] for 768-dimensional embeddings, with Tesseract OCR [24] for scanned papers. The data layer holds the PDF library, per-subject FAISS [21] indexes, a question bank of 1,426 questions, an answer cache, curriculum questions and repeated-question solutions.

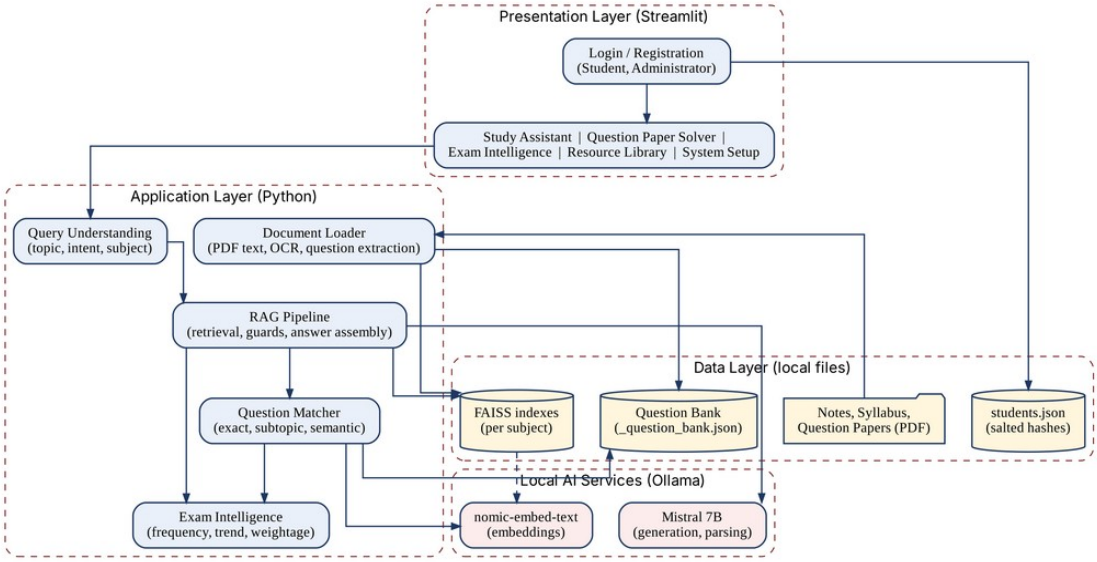


Fig. 1. System architecture of uniRAG.

A question follows the pipeline in Fig. 2: it is parsed into structured fields, checked against the syllabus gate, and then passed to both the notes retriever and the question matcher. The retrieved notes chunks and matched past questions are combined in a prompt, the local model streams a two-part answer (concept and analogy), deterministic exam facts (frequency, years, marks, priority) are attached, and the result is cached.

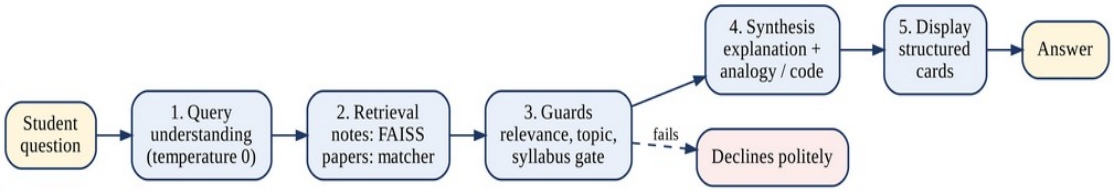


Fig. 2. Answer pipeline.

IV. IMPLEMENTATION

A. Technology stack

Table II shows the components. All software is open source, runs locally and needs no cloud account or API key.

TABLE II. Technology stack of uniRAG.

Component	Tool	Role
Interface	Streamlit	Login, tabs, chat, uploads, charts
Models	Ollama; Mistral 7B	Parsing, explanations, lab code
Embeddings	nomic-embed-text	768-d vectors for notes and questions
Search	FAISS (faiss-cpu)	Per-subject vector indexes
Text splitting	LangChain	400-char chunks, 50 overlap
PDF / OCR	PyPDF, PyMuPDF, Tesseract	Text layer, rendering, scanned pages
Charts	pandas, Altair	Trend and ranking charts
Security	hashlib, hmac, secrets	Salted PBKDF2 hashes, lockout

B. Modules and data stores

The code is organised into modules for the user interface (app.py), configuration, document loading, query understanding, the RAG pipeline, the question index, the question matcher, exam intelligence, curriculum questions and repeated-question solutions. Structured data is kept in JSON files (student accounts, the question bank, a build manifest, the answer cache, the solutions store and university settings), while embeddings are kept in FAISS indexes, one per subject plus one for the questions. For this workload of about 1,400 questions written in bulk by a rebuild and then only read, JSON and FAISS need no database server, can be copied with the project folder and make full rebuilds simple.

C. Application features

A student uses four tabs. The Study Assistant answers conceptual questions with a textbook explanation, a plain-language analogy and the exam history of the topic (Fig. 3). The Question Paper Solver accepts a whole paper, extracts its questions, answers each one in turn and offers a combined PDF download. The Exam Intelligence tab displays the exam history of a topic (Fig. 4), its year-by-year trend, the most repeated questions with a solutions PDF and the high-weightage questions for a chosen marks level. The Resource Library lists notes, syllabus and question papers by semester and subject. An administrator tab lets faculty upload PDFs, rebuild indexes safely and re-configure university settings.

V. METHODOLOGY

A. Document processing and question bank

Notes and syllabi are chunked into 400-character blocks with 50-character overlap and embedded into one FAISS index per subject; a single merged index mixed subjects and let large subjects crowd out small ones, so it is kept only as a fallback. Question papers are processed by an extractor that uses regular expressions and Tesseract OCR to record individual questions with subject, semester, year and marks. Scanner noise, subpart joins, missing question numbers and scheme boilerplate are cleaned by rules. The outcome is a question bank of 1,426 questions from 18 subjects across seven exam years (2020–2026), together with a small vector index of the questions.

B. Query understanding

A small prompt to Mistral 7B converts the student's raw text into a JSON object with intent (definition, frequency, exact questions, trend, ranking, lab program and so on), clean topic and subject. Code-level rules correct common mistakes: follow-up topics are only accepted if the question has a pronoun reference, and Hinglish or Kannada words are mapped to their English syllabus terms.

C. Hybrid question matching

Questions are matched at four levels: exact, subtopic, semantic (a dedicated question vector store) and filters on marks and year. A fuzzy match uses the difflib ratio, where M is the number of matching characters and T the total number of characters in both strings:

$$ratio(A, B) = 2M / T \qquad (1)$$

A ratio of at least 0.75 counts as a near-exact match, above 0.8 as a subtopic match. For semantic matching, the query is embedded and compared with the question vector store using FAISS.

The FAISS flat index returns the squared Euclidean distance between the 768-dimensional normalised embeddings of query q and stored question v:

$$d^2(q, v) = \sum_{i=1..768} (q_i - v_i)^2 \qquad (2)$$

Because vectors are L2-normalised, squared Euclidean distance is related to cosine similarity by $d^2 = 2(1 - \cos \theta)$. A distance cut-off decides whether a candidate is relevant.

D. Guards

Table III lists the seven code-level guards that protect against hallucination. Out-of-syllabus questions never reach the model, so they are answered instantly (0 s).

TABLE III. Guards against wrong answers.

Guard	What it stops
Relevance cut-off (distance ≤ 1.15)	Answers built from unrelated text
Topic-in-context check	A topic answered from model memory
Out-of-syllabus gate	Definitions of topics absent from syllabus and papers
Deterministic counting	Invented frequencies or years
Follow-up check	Topic carried into an unrelated question
Figure-reference stripping	“Figure 3.1 shows…” the student cannot see
Empty-input guard	A bare “?” giving a random answer

E. Deterministic exam analytics

Counts, years and marks are read from the question bank. The year-by-year trend of a topic is classified by splitting the yearly counts into an earlier and a later half and comparing their averages:

$$\textit{rising if avg(later)} > \textit{avg(earlier)} + 0.5 \qquad (3)$$

with falling for the symmetric case and stable otherwise. Topic coverage over the exam period is:

$$\textit{Coverage (\%)} = 100 \times D / Y \qquad (4)$$

where D is the number of distinct exam years in which the topic appeared and Y is the total number of exam years for the subject. Questions asked in two or more sittings are listed as the most repeated questions, and a PDF solutions bundle is generated for them.

F. Answer cache and security

A saved answer is looked up by a SHA-256 digest of question, subject and retrieved context: if an update changes the retrieved notes, it also changes the key. Accounts use PBKDF2-HMAC-SHA256 with 100,000 iterations and a unique salt per account, compared in constant time. Five failed passwords lock an account for 60 s.

VI. EXPERIMENTAL SETUP

All measurements were made on a Lenovo IdeaPad Gaming 3 laptop (Intel Core i5-11300H at 3.10 GHz, 4 cores, 8 threads, 16 GB DDR4 RAM, 512 GB NVMe SSD, Windows 11 Home, Python 3.12.7). The models were mistral:7b (answer temperature 0.1, parsing temperature 0) and nomic-embed-text, chunk size 400, overlap 50 and top-k 3. No external GPU or network was used.

The question-matching cut-off was calibrated on a labelled set of 61 queries (46 on-syllabus and 15 out-of-syllabus or off-topic) across the lecture notes and question papers of 18 subjects of the first three semesters. For each candidate value of the distance cut-off from 0.30 to 1.20 the returned questions were compared with the manual labels to compute precision (P), recall (R) and F1 score, defined in terms of the numbers of correct questions returned, wrong questions returned and correct questions missed:

$$P = TP / (TP + FP), \quad R = TP / (TP + FN) \qquad (5)$$

$$F1 = 2PR / (P + R) \qquad (6)$$

A regression suite of 29 fixed cases with 44 checks, run by eval_questions.py, verified the guards (intent, topic bleeding, out-of-syllabus refusal, lab language, determinism and count consistency across repeated questions); it also records the response time of each question type.

VII. RESULTS AND DISCUSSION

A. Question bank

The question bank holds 1,426 questions: 692 carry 6 to 10 marks (high weightage), 565 carry 3 to 5 marks and 169 carry 2 marks (Fig. 5). The exam years 2022 to 2025 hold 274 to 326 questions each, while 2020 and 2026 hold fewer (Fig. 6); Bangalore University conducted fewer papers during 2020, and 2026 had only early-session papers at the time of collection, so those two years rest on fewer papers. Subjects differ widely in size, from 118 questions (QTR) to 14 (CNS), so counts for small subjects rest on a small sample (Fig. 7). Across 16 subjects, 125 questions were repeated in two or more sittings.

The number of questions per exam year also differs between semesters: every semester has papers from five or six of the seven years, Semester 1 has no 2026 papers, Semester 2 has no 2021 or 2026 papers, and Semester 3 has no 2020 papers but is the only semester with 2026 papers. Because of these gaps, uniRAG calculates a topic's exam coverage using only the years in which that subject actually had a paper, not a flat seven years.

B. Matching quality

Table IV and Fig. 8 show the expected trade-off. A strict cut-off of 0.50 gives almost perfect precision but misses most related questions (recall 0.298); the original permissive value of 1.15 finds most related questions (recall 0.917) but shows 665 wrong questions across the test set and answers all 15 queries that should match nothing. The peak F1 score of 0.757 occurs at 0.75 (precision 0.777, recall 0.737, 44 of 46 on-syllabus queries hit, only 1 of 15 nothing-queries answered wrongly). Setting the cut-off to 0.75 cut false-positive questions by a factor of eleven compared with 1.15.

TABLE IV. Matching quality at selected cut-offs (61 queries).

Cut-off	Precision	Recall	F1	Nothing-queries wrongly answered
0.50	0.989	0.298	0.457	0/15
0.60	0.907	0.474	0.623	0/15
0.70	0.855	0.651	0.739	1/15
0.75	**0.777**	**0.737**	**0.757**	**1/15**
0.90	0.490	0.879	0.629	7/15
1.15	0.285	0.917	0.435	15/15

C. Response time

Table V and Fig. 10 report measured response times across the 29 regression cases.

TABLE V. Response time by question type (CPU only).

Question type	Runs	Median (s)	Range (s)
Out-of-syllabus (declined)	3	0	0
Exam facts	11	25	23–112
Explanation of a topic	12	77	51–214
Comparison of two topics	2	131	104–157
Lab program	2	193	173–212

Declined questions are instant because the guard stops them before any model call, and exam-fact questions take about 25 s because the model only parses the question while code produces the counts. Explanations and lab programs take one to three and a half minutes because the model must generate long text on a CPU. A cached answer returns in seconds. These times are slower than the 3–6 s reported for a larger model on an Apple-silicon laptop [12], which reflects the Intel CPU and the long, structured answers; the persisted indexes and the answer cache play the same latency-saving role as the persisted index reported in [13].

D. Repeated questions and solutions bundle

The Most Repeated Questions view lists questions asked in two or more sittings of a subject and bundles a model answer for each into one PDF. Of the 125 repeated questions, 67 have curated answers (drafted with an AI assistant, checked, and numerical results recomputed in code) and 58 were generated offline by the local 7B model from the notes (Fig. 9). The curated answers avoid the calculation and proof errors that the local 7B model made on numerical subjects such as QTR and DAA.

E. Functional and regression testing

The 27 manual test cases are grouped in Table VI. The 29 automated cases in eval_questions.py were run after every code change; they caught seven regressions during development, including a false-pass bug where an intent rule matched the rule name in its own output, and an ungrounded-answer bug on out-of-syllabus topics.

TABLE VI. Manual test cases by area.

Area	Cases	Example check
Accounts and roles	7	Lockout after five wrong passwords
Study questions	11	Out-of-syllabus topic is declined
Exam Intelligence	4	Trend needs a selected subject
Question Paper Solver	1	Questions extracted and answered
OCR	1	Scanned page read with OCR
Repeatability	1	Same question, same answer
Rebuild and data safety	2	Corrupt file backed up

F. Worked examples

Exam history. For the question “what is queue? linear and circular, how many times it appeared” with the subject DS selected, uniRAG reported that the topic appeared in 5 sittings (6 questions) in 2025, 2024, 2023, 2022 and 2020, a coverage of 100 * 5/6 ≈ 83 %, listed the six questions with year and marks, and gave the priority tier “High exam priority (repeated topic)”. The numbers were identical on repeated runs.

Year-by-year trend. For the topic “Queue” the trend showed 2 matching questions in 2020, none in 2021 and one in each of 2022 to 2025, classified as stable. An early version showed the Data Structures chart when Computer Networks was selected, because the matcher fell back to the whole bank; the trend now uses only the selected subject.

Out-of-syllabus questions. “What is a Fenwick tree”, “explain Kubernetes in cloud” and “what is backpropagation” (when asked under MCA semester 1) were all declined immediately with a clear message stating that the topic does not appear in the MCA syllabus or past papers.

G. Discussion

Table VII compares uniRAG with a general chatbot.

TABLE VII. uniRAG compared with a general chatbot.

Aspect	General chatbot	uniRAG

Internet	Required	**Not needed**
Privacy	Questions sent out	**Data stays local**
Answer source	General knowledge	**University notes and papers**
Out of syllabus	Answered	**Declined**
Exam statistics	Manual or invented	**Counted by code**
Scanned papers	Read by hand	**OCR to question bank**
Cost	Subscription or API fee	**Free, open source**

Separating the work of the model from the work of the code gave numbers that are exact, repeatable and traceable to the question bank, which a general chatbot cannot guarantee. The price is that counting is literal: a topic worded differently in a paper can be missed, a deliberate trade-off against invented counts. In the solutions bundle, the local model was unreliable on calculation and proof questions, since 41 of its first 99 answers had to be replaced, so descriptive explanations are more reliable than numerical ones. This agrees with the general finding that retrieval reduces but does not remove hallucination [4], and it is why uniRAG lists the source files with every answer and treats answers as study aids.

VIII. LIMITATIONS AND THREATS TO VALIDITY

The system was evaluated on one programme of one university (Bangalore University MCA, semesters 1 to 3); other programmes and universities may have different paper structures, languages and grading schemes. The question-matching cut-off was calibrated on 61 queries labelled by one author; a multi-annotator evaluation would be more reliable. OCR quality on older, handwritten or low-resolution papers is imperfect, which explains why six subjects required manual correction in the question bank. Although the 29-case regression harness verifies that guards trigger correctly, answer factuality was evaluated qualitatively rather than with human-annotated ground-truth answers. Finally, CPU inference on a four-core laptop requires 50 to 200 s for generative answers, which limits throughput unless GPU acceleration or a smaller parsing model is used.

IX. CONCLUSION AND FUTURE WORK

This paper presented uniRAG, an offline retrieval-augmented study assistant that runs on a single consumer laptop and combines course-note explanations with deterministic exam intelligence for university students. By delegating text generation to a local Mistral 7B model and all counting, trending and ranking to Python code over a structured question bank, the system eliminates the invented frequencies typical of general chatbots while protecting student privacy and keeping operating costs at zero. A calibrated relevance cut-off (0.75) improved matching F1 from 0.435 to 0.757 and cut false positives by a factor of eleven. A suite of seven code-level guards, including an out-of-syllabus gate and an answer cache, ensures that out-of-scope questions are declined immediately and that repeated queries return in seconds.

Future work will investigate faster inference using GPU acceleration and smaller specialised models for parsing, automatic diagram and table extraction from question papers [19], support for Kannada and Hindi questions using multilingual embedders [20], a multi-user campus deployment with email verification, and a student user study measuring the impact on examination performance.

REFERENCES

[1] Ministry of Education, Government of India, All India Survey on Higher Education 2021-22. [Online]. Available: https://education.gov.in/sites/upload_files/mhrd/files/statistics-new/AISHE%20Book_2021-22_4.pdf

[2] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” arXiv:2005.11401, 2020.

[3] Y. Gao et al., “Retrieval-augmented generation for large language models: A survey,” arXiv:2312.10997, 2023.

[4] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” ACM Trans. Inf. Syst., arXiv:2311.05232, 2025.

[5] Z. Ji et al., “Survey of hallucination in natural language generation,” ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023.

[6] A. Asai, Z. Wu, Y. Wang, A. Sil and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in Proc. ICLR, 2024, arXiv:2310.11511.

[7] S. Es, J. James, L. Espinosa-Anke and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” arXiv:2309.15217, 2023.

[8] X. Li, Y. Cao, Y. Ma and A. Sun, “Long context vs. RAG for LLMs: An evaluation and revisits,” arXiv:2501.01880, 2024.

[9] G. Lang and T. Gurpinar, “AI-powered learning support: A study of retrieval-augmented generation (RAG) chatbot effectiveness in an online course,” Inf. Syst. Educ. J., vol. 23, no. 2, pp. 4–13, Mar. 2025.

[10] S. Kahl et al., “Evaluating the impact of advanced LLM techniques on AI-lecture tutors for a robotics course,” arXiv:2408.04645, 2024.

[11] A. Ostrowska et al., “From surface learning to deep understanding: A grounded AI tutoring system for Moodle,” arXiv:2605.06963, 2026.

[12] T. Takamura and A. Umezawa, “Privacy-preserving retrieval-augmented generation on local devices for regenerative medicine applications,” medRxiv, doi: 10.1101/2025.10.20.25337146, 2025.

[13] A. S. Lee, “Offline RAG with small language models optimized for resource-constrained environments: Natural disaster safety advice,” poster, NeurIPS 2026 Affinity Event, 2026.

[14] P. Devine, “ALoFTRAG: Automatic local fine tuning for retrieval augmented generation,” arXiv:2501.11929, 2025.

[15] R. Dayarathne, U. Ranaweera and U. Ganegoda, “Comparing the performance of LLMs in RAG-based question-answering: A case study in computer science literature,” arXiv:2511.03261, 2025.

[16] Q. Luo, X. Li, T. Fan, X. Chen and X. Qiu, “Towards global retrieval augmented generation: A benchmark for corpus-level reasoning,” arXiv:2510.26205, 2025.

[17] H. Zhu et al., “Aggregation queries over unstructured text: Benchmark and agentic method,” arXiv:2602.01355, 2026.

[18] J. Zhang et al., “OCR hinders RAG: Evaluating the cascading impact of OCR on retrieval-augmented generation,” arXiv:2412.02592, 2025.

[19] L. Sun et al., “When good OCR is not enough: Benchmarking OCR robustness for retrieval-augmented generation,” arXiv:2605.00911, 2026.

[20] P. Prasanjith, P. B. More, A. Kunchukuttan and R. Dabre, “IndicRAGSuite: Large-scale datasets and a benchmark for Indian language RAG systems,” arXiv:2506.01615, 2025.

[21] J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” arXiv:1702.08734, 2017.

[22] Z. Nussbaum et al., “Nomic Embed: Training a reproducible long context text embedder,” arXiv:2402.01613, 2024.

[23] A. Q. Jiang et al., “Mistral 7B,” arXiv:2310.06825, 2023.

[24] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[25] K. Moriarty, B. Kaliski and A. Rusch, “PKCS #5: Password-based cryptography specification version 2.1,” RFC 8018, IETF, 2017.